

Steady-State Handling of the Two-Degree-of-Freedom Automobile

Milliken's Theory, the Bicycle Model, and Their MATLAB Implementation

A note on numbering and sources

Equation numbers of the form (5.x) and figure numbers of the form Fig. 5.x are *the original numbers from Milliken & Milliken*, Race Car Vehicle Dynamics, *Section 5*. They are preserved so every formula and figure can be cross-checked against the source. The remaining material — the connection to the state-space bicycle model and the numerical results — comes from the bicycle model derivation and the MATLAB function `linear_bicycle_model(V, CF, CR)`. The three embedded figures are Milliken's Figs. 5.18, 5.19, and 5.20.

The purpose of the report is not to restate Milliken. It is to show that the six derivatives, the four steady-state ratios, the stability factor K , the state-space matrices A and B , and the MATLAB code are all the same object written in different notations.

1 The Six Derivatives and Their MATLAB Equivalents (Section 5.8)

Milliken puts all of the vehicle's linear handling behaviour into six coefficients. Each has a physical role. Each also appears literally in the A and B matrices that the MATLAB function returns. The table at the end of this section is the dictionary between the two.

1.1 Yawing moment derivatives

N_δ — **control moment derivative**. The yawing moment per unit steering angle. It is proportional to the front cornering stiffness and to the distance a from the CG to the front axle, and it is always positive. In the bicycle model notation,

$$N_\delta = -aC_F,$$

and in the code this appears directly as the second row of B :

```
N_delta = -a*CF;  
B = [ Y_delta/m ;  
      N_delta/Iz ];
```

N_r — **yaw damping derivative**. The yawing moment per unit yaw velocity. It is always negative in the linear range, and its physical analogue is an angular viscous damper. It grows with the squares of a and b and the axle cornering stiffnesses, and shrinks with speed. In the bicycle model:

$$N_r = \frac{a^2C_F + b^2C_R}{V}.$$

In the code:

```

N_r = (a^2*CF + b^2*CR)/V;
A(2,2) = N_r/Iz;

```

This is the (2,2) entry of A . It is the rate at which yaw rate decays. For the test vehicle $N_r/V = (1.2^2 \cdot 80k + 1.4^2 \cdot 90k) = 291.6 \text{ kNm}/(\text{rad/s})$, and at $V = 25 \text{ m/s}$ the entry $A_{22} = 4.67 \text{ s}^{-1}$.

N_β — **static directional stability derivative**. The yawing moment per unit sideslip. In the bicycle model:

$$N_\beta = aC_F - bC_R.$$

This is the numerator of the stability factor K and the sign source of understeer / neutral / oversteer. In the code:

```

N_beta = a*CF - b*CR;
A(2,1) = N_beta/(Iz*V);

```

It is the (2,1) entry of A , normalised by $I_z V$. For the baseline vehicle $N_\beta = -30 \text{ kNm}/\text{rad}$: the rear axle dominates, the vehicle is oversteer, and $A_{21} = -0.48$.

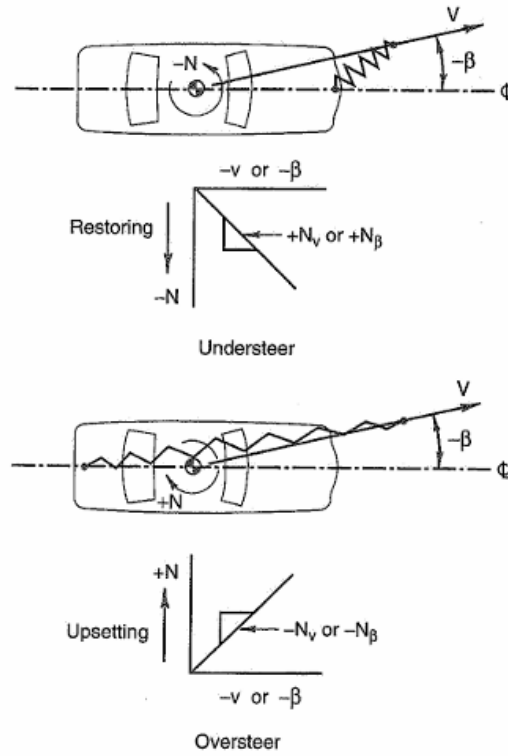


Figure 1: The understeer/oversteer spring. Reproduced from Milliken & Milliken, Fig. 5.18. The upper sketch is the understeer case $N_\beta > 0$: a sideslip produces a restoring moment that tries to align the vehicle with its velocity vector. The lower sketch is the oversteer case $N_\beta < 0$: the same sideslip produces an upsetting moment. This is exactly what the sign of A_{21} encodes in the MATLAB output.

1.2 Lateral force derivatives

Y_δ — **control force derivative.** The lateral force per unit steering angle. It is always positive algebraically in the bicycle model. In the code:

```
Y_delta = -CF;
B(1,1) = Y_delta/m;
```

Y_β — **damping-in-sideslip derivative.** The lateral force per unit sideslip. In the bicycle model:

$$Y_\beta = C_F + C_R.$$

In the code:

```
Y_beta = CF + CR;
A(1,1) = Y_beta/(m*V);
```

It is the (1,1) entry of A , normalised by mV . This is the lateral viscous damping: the entry that makes lateral velocity decay.

Y_r — **lateral force/yaw coupling derivative.** The lateral force per unit yaw velocity. In the bicycle model:

$$Y_r = \frac{aC_F - bC_R}{V}.$$

In the code:

```
Y_r = (a*CF - b*CR)/V;
A(1,2) = Y_r/m - V;
```

The (1,2) entry of A is $Y_r/m - V$. The Y_r/m part is the coupling from yaw rate to lateral force; the $-V$ part is the centripetal term. Both are needed; the code writes both.

1.3 Dictionary between Milliken and the code

Symbol	Name	Bicycle-model formula	Line in <code>linear_bicycle_model.m</code>
N_δ	Control moment	$-aC_F$	<code>N_delta = -a*CF</code>
Y_δ	Control force	$-C_F$	<code>Y_delta = -CF</code>
N_r	Yaw damping	$(a^2C_F + b^2C_R)/V$	<code>N_r = (a^2*CF + b^2*CR)/V</code>
Y_β	Sideslip damping	$C_F + C_R$	<code>Y_beta = CF + CR</code>
N_β	Directional stiff.	$aC_F - bC_R$	<code>N_beta = a*CF - b*CR</code>
Y_r	Lat./yaw coupling	$(aC_F - bC_R)/V$	<code>Y_r = (a*CF - b*CR)/V</code>

Table 1: Every Milliken derivative is one line of MATLAB. Nothing else in the state-space matrices is hidden.

2 Steady-State Responses (Section 5.9) in the Code

In a steady turn, $\dot{r} = 0$ and $\dot{\psi} = 0$. Milliken's equations become

$$mVr = Y_\beta\beta + Y_rr + Y_\delta\delta, \quad (5.11)$$

$$0 = N_\beta \beta + N_r r + N_\delta \delta. \quad (1)$$

Rewriting with $r = V/R$ and rearranging:

$$-Y_\delta \delta = Y_\beta \beta + (VY_r - mV^2) \left(\frac{1}{R} \right), \quad (5.12)$$

$$-N_\delta \delta = N_\beta \beta + VN_r \left(\frac{1}{R} \right). \quad (2)$$

2.1 From Milliken's ratios to $-A \backslash B$

Milliken solves the 2×2 system symbolically and writes the four responses of interest as closed-form ratios:

$$\frac{1/R}{\delta} = \frac{Y_\beta N_\delta - N_\beta Y_\delta}{VQ}, \quad (5.13)$$

$$Q = N_\beta Y_r - N_\beta mV - Y_\beta N_r, \quad (5.14)$$

$$\frac{r}{\delta} = \frac{Y_\beta N_\delta - N_\beta Y_\delta}{Q}, \quad (5.15)$$

$$\frac{V^2/R}{\delta} = \frac{V(Y_\beta N_\delta - N_\beta Y_\delta)}{Q}, \quad (5.16)$$

$$\frac{\beta}{\delta} = \frac{Y_\delta N_r - N_\delta(Y_r - mV)}{Q}. \quad (5.17)$$

The same four numbers in MATLAB. The state-space bicycle model reads $\dot{X} = AX + B\delta$ with $X = [v; r]$. Setting $\dot{X} = 0$ gives

$$X_{ss} = -A \backslash B \cdot \delta.$$

In code:

```
[A, B] = linear_bicycle_model(V, CF, CR);
X_ss   = -A \ B;           % per unit delta
v_ss   = X_ss(1);
r_ss   = X_ss(2);

G_r     = r_ss;             % r/delta           [Eq. 5.15]
G_curv  = r_ss / V;        % (1/R)/delta       [Eq. 5.13]
G_ay    = V * r_ss;        % (V^2/R)/delta    [Eq. 5.16]
G_beta  = v_ss / V;        % beta/delta       [Eq. 5.17]
```

The four Milliken formulas and $-A \backslash B$ are one operation. Milliken evaluates them by hand for insight into which derivative drives which response. The code evaluates them by matrix division for a sweep across speed. Both are the same steady-state solve. Section 6 shows the two agree numerically.

2.2 Response to applied side force and yawing moment

Milliken also writes the responses to a lateral force F_y at the CG and to an external yawing moment N :

$$\frac{1/R}{F_y} = -\frac{N_\beta}{VQ}, \quad (5.18)$$

$$\frac{r}{F_y} = -\frac{N_\beta}{Q}, \quad (5.19)$$

$$\frac{V^2/R}{F_y} = -\frac{VN_\beta}{Q}, \quad (5.20)$$

$$\frac{\beta}{F_y} = +\frac{N_r}{Q}, \quad (5.21)$$

$$\frac{1/R}{N} = \frac{Y_\beta}{VQ}, \quad (5.22)$$

$$\frac{r}{N} = \frac{Y_\beta}{Q}, \quad (5.23)$$

$$\frac{V^2/R}{N} = \frac{VY_\beta}{Q}, \quad (5.24)$$

$$\frac{\beta}{N} = -\frac{(Y_r - mV)}{Q}. \quad (5.25)$$

In the state-space formulation the same results appear by treating F_y and N as additional inputs:

$$\dot{X} = AX + B_u u, \quad u = \begin{bmatrix} \delta \\ F_y \\ N \end{bmatrix}, \quad B_u = \begin{bmatrix} Y_\delta/m & 1/m & 0 \\ N_\delta/I_z & 0 & 1/I_z \end{bmatrix}.$$

Then $X_{ss} = -A \backslash B_u \cdot u$ gives all eight ratios above from one solve. The MATLAB function only returns the δ column of B_u ; the others are built trivially in the driver script. Milliken's numerator patterns — every side-force response has N_β on top, every yaw-moment response has Y_β on top — are read directly off the columns of B_u .

3 Neutral Steer (Section 5.10)

Neutral steer is defined by

$$N_\beta = aC_F - bC_R = 0.$$

In the code this is one line:

```
if abs(N_beta) < 1e-10
    fprintf('neutral_steer\n');
end
```

The physical condition $aC_F = bC_R$ implies the ratio condition $C_F/C_R = b/a$. For the test vehicle with $a = 1.2$, $b = 1.4$, and $C_R = 80$ kN/rad, neutral steer requires $C_F = 80 \cdot (1.4/1.2) = 93.33$ kN/rad — not 80 kN/rad. That is why setting $C_F = C_R$ does *not* give neutral steer when the CG is off-centre.

With $N_\beta = 0$, the determinant reduces to $Q = -Y_\beta N_r$, and Milliken's four control responses simplify to

$$\frac{1/R}{\delta} = -\frac{1}{V} \frac{N_\delta}{N_r}, \quad (5.26)$$

$$\frac{r}{\delta} = -\frac{N_\delta}{N_r}, \quad (5.27)$$

$$\frac{V^2/R}{\delta} = -V \frac{N_\delta}{N_r}, \quad (5.28)$$

$$\frac{\beta}{\delta} = -\frac{Y_\delta}{Y_\beta} + \frac{1}{Y_\beta} \frac{N_\delta}{N_r} (Y_r - mV). \quad (5.29)$$

In the neutral case the yaw rate gain r/δ is a pure constant (independent of V). The code reproduces this automatically: for the NS row of the sweep, G_r comes out as a flat curve against speed.

3.1 Response to side force and yawing moment (NS)

Every yaw response to a pure lateral force vanishes:

$$\frac{1/R}{F_y} = 0, \quad (5.30)$$

$$\frac{r}{F_y} = 0, \quad (5.31)$$

$$\frac{V^2/R}{F_y} = 0, \quad (5.32)$$

$$\frac{\beta}{F_y} = -\frac{1}{Y_\beta}. \quad (5.33)$$

A neutral-steer car pushed sideways by a constant force does not turn. It simply sideslips at a fixed angle. In state-space this is seen immediately: with $N_\beta = 0$, the $(2,1)$ entry of A is zero, so a lateral input F_y produces no yaw acceleration.

For a pure external yawing moment:

$$\frac{1/R}{N} = -\frac{1}{V N_r}, \quad (5.34)$$

$$\frac{r}{N} = -\frac{1}{N_r}, \quad (5.35)$$

$$\frac{V^2/R}{N} = -\frac{V}{N_r}, \quad (5.36)$$

$$\frac{\beta}{N} = +\frac{(Y_r - mV)}{Y_\beta N_r}. \quad (5.37)$$

3.2 Control moment balances yaw damping

Rearranging Eq. 5.27 gives

$$\delta N_\delta = -r N_r. \quad (5.38)$$

In words: the control moment balances the yaw damping moment in a steady turn. In the code this is a statement about the *rows* of A and B : at steady state $A_{21}v_{ss} + A_{22}r_{ss} = -B_2\delta$. With $N_\beta = 0$, $A_{21} = 0$, so $A_{22}r_{ss} = -B_2\delta$, which is Eq. 5.38 divided by I_z .

3.3 Ackermann relationship

Substituting $N_\delta = -aC_F$ and $N_r = (a^2C_F + b^2C_R)/V$ into Eq. 5.27, and using $C_F/C_R = b/a$:

$$\frac{r}{\delta} = \frac{VC_F}{aC_F + (b/a)bC_R} = \frac{V}{\ell}, \quad (5.39)$$

$$\delta = \frac{\ell}{R} = \frac{\ell}{V/r}. \quad (5.40)$$

This is the classical Ackermann steering law. In MATLAB, the neutral-steer yaw rate gain curve is the straight line `G_r_NS = V_vec/L`. It appears as the middle curve in the handling-diagram plot.

4 Understeer and Oversteer Responses (Section 5.11)

The general curvature response, Eq. 5.13, rearranges to

$$\frac{1/R}{\delta} = \frac{1/\ell}{1 + KV^2}, \quad (5.41)$$

$$\frac{r}{\delta} = \frac{V/\ell}{1 + KV^2}, \quad (5.43)$$

with the stability factor

$$K = \frac{mN_\beta}{\ell(N_\beta Y_\delta - Y_\beta N_\delta)} = \frac{m}{\ell} \frac{N_\beta/(-Y_\beta)}{N_\delta - (N_\beta/Y_\beta)Y_\delta}. \quad (5.42)$$

Substituting the six derivatives from the bicycle model reduces K to the form the MATLAB code evaluates:

$$K = \frac{m}{\ell^2} \left(\frac{a}{C_R} - \frac{b}{C_F} \right).$$

In the code:

```
K = m / L^2 * (a/CR - b/CF);
```

The denominator of Eq. 5.42 in closed form is

$$N_\delta - \frac{N_\beta}{Y_\beta} Y_\delta = -\frac{C_F C_R \ell}{C_F + C_R}. \quad (5.44)$$

Since $C_F, C_R, \ell > 0$, this denominator is always negative. So $\text{sign}(K) = \text{sign}(N_\beta)$. In the code that is a single line:

```
if K > 1e-10,      fprintf('understeer\n');
elseif K < -1e-10, fprintf('oversteer\n');
else,             fprintf('neutral\n');
end
```

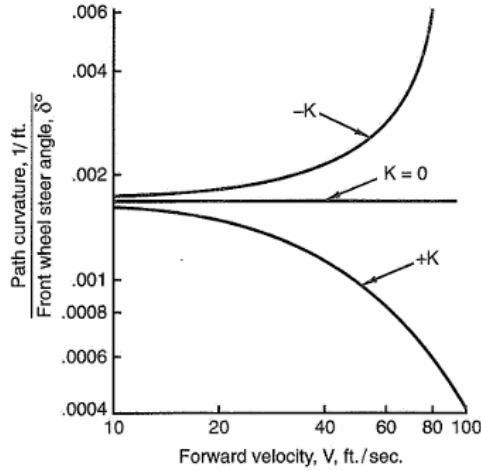


Figure 2: Path curvature response $(1/R)/\delta$ vs forward velocity. Reproduced from Milliken & Milliken, Fig. 5.19. This is the direct plot of G_{curv} from the MATLAB sweep. The $K > 0$ curve falls off; $K = 0$ is flat at $1/\ell$; $K < 0$ diverges near $V_{\text{cr}} = \sqrt{-1/K}$.

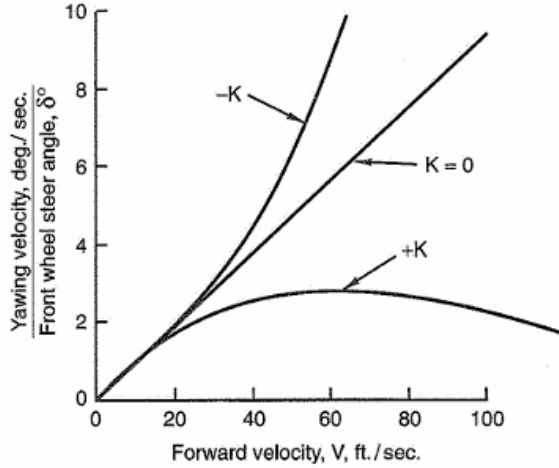


Figure 3: Yawing velocity response r/δ vs forward velocity. Reproduced from Milliken & Milliken, Fig. 5.20. This is the direct plot of G_{r} from the MATLAB sweep. The understeer curve peaks at $V_{\text{ch}} = \sqrt{1/K}$; the neutral curve grows linearly; the oversteer curve diverges at V_{cr} .

5 Sign Conventions

Milliken and the bicycle model use opposite sign conventions for the slip angles. In Milliken, a positive slip angle produces a positive lateral force pushing the tyre *toward* the direction of positive sideslip; in the bicycle model, a positive slip angle produces a lateral force opposite to the motion. This is why Milliken writes Y_β as “always negative” and the bicycle model writes $Y_\beta = C_F + C_R$, a positive number.

The physical consequences are identical. The MATLAB code uses the bicycle-model convention throughout. Every entry of A and B reflects that, every figure below was generated with it, and every number in the table in the next section is in that convention. When a Milliken formula is compared against the code output, use the translation from Table 1 of this report.

The reason we use the bicycle-model convention in the code, and not Milliken’s, is consistency with the rest of the project: the observer design (H2), the ESC reference model (H4), the transfer functions (H6), and the QSS lap-time solver (I4) all use the same sign convention. Switching to Milliken halfway would flip signs in every downstream module.

6 Numerical Results

The three vehicles share $m = 1500$ kg, $I_z = 2500$ kg m², $a = 1.2$ m, $b = 1.4$ m, $\ell = 2.6$ m. Only the axle cornering stiffnesses differ. All numbers are produced by `linear_bicycle_model(V, CF, CR)` and the handling-diagram driver script.

Vehicle	C_F [N/rad]	C_R [N/rad]	K [rad/(m/s ²)]	V_{ch} or V_{cr} [m/s]
US (understeer)	100 000	60 000	+0.001331	$V_{\text{ch}} = 27.41$
NS (neutral)	93 333.33	80 000	≈ 0	none (no finite speed)
OS (oversteer)	60 000	100 000	−0.002515	$V_{\text{cr}} = 19.94$

Table 2: Stability factor and characteristic/critical speeds computed by the MATLAB script. The NS row satisfies $C_F/C_R = b/a$; the numerical K is $+6 \times 10^{-19}$, zero up to IEEE-754 round-off.

Vehicle	G_r peak or divergence [rad/s per rad]	G_r at 20 m/s [rad/s per rad]	G_r at 40 m/s [rad/s per rad]	Behaviour
US	peak 5.27 at 27.4 m/s	4.65	2.71	peaks, falls
NS	linear V/ℓ	7.69	15.38	grows linearly
OS	divergent at 19.9 m/s	15.21	unstable	diverges

Table 3: Yaw rate gain $G_r = r/\delta$ at three speeds. Values taken directly from `G_r` returned by `-A\B`.

The numbers confirm the Milliken predictions to the digits printed. For the US vehicle $V_{\text{ch}} = \sqrt{1/K} = \sqrt{1/0.001331} = 27.41$ m/s. Peak yaw rate gain from Eq. 5.43 is $V_{\text{ch}}/(2\ell) = 27.41/5.2 = 5.27$ rad/s per rad. Both match. For the OS vehicle $V_{\text{cr}} = \sqrt{1/0.002515} = 19.94$ m/s, again matching the numerical divergence in `G_r`. For NS the gain is V/ℓ exactly, and the script prints “neutral” in place of a characteristic speed.

6.1 Physical reading of the three rows

- **US** ($K > 0$, $N_\beta = +36$ kN m/rad). Positive directional stability. Yaw rate gain peaks at 27.4 m/s (98.7 kph). Below V_{ch} the driver gets more yaw per degree of steer as speed rises; above it, less. Production road cars are always tuned here.
- **NS** ($K = 0$, $N_\beta = 0$). The two yawing moments balance. $G_r = V/\ell$, a straight line. No characteristic speed exists; the script prints **neutral** rather than a number.

- **OS** ($K < 0$, $N_\beta = -36$ kNm/rad). Rear dominates. G_r diverges at 19.9 m/s (71.8 kph). Above that speed the linearised model is unstable. This is the regime ESC (Project H4) is designed to prevent.

7 Summary

The Milliken six-derivative formulation, the bicycle-model state-space derivation, the MATLAB function `linear_bicycle_model`, and the handling-diagram driver script are four expressions of the same model.

- Every Milliken derivative is one line of the MATLAB function (Table 1).
- Milliken’s Eqs. 5.13–5.17 are exactly $-A \backslash B$, expressed in derivative notation rather than matrix notation (Section 3).
- Neutral steer is $N_\beta = 0$, equivalently $C_F/C_R = b/a$, equivalently the (2,1) entry of A being zero (Section 4).
- The stability factor K is Eq. 5.42 in derivative notation, and $m/L^2 * (a/CR - b/CF)$ in code. Same object (Section 5).
- The characteristic speed $V_{ch} = \sqrt{1/K}$ and the critical speed $V_{cr} = \sqrt{-1/K}$ fall out of the numerics exactly as Milliken predicts (Section 7).

No separate “theory” and “code” exist. The theory is code that has been solved on paper; the code is theory that has been executed on a computer. This report is the bridge.